Evaluation Strategies (Workshop)

Concepts of Programming Languages

CAS CS 320

Recap

Lambda Calculus (Syntax)

$$e ::= \lambda x \,.\, e \mid ee \mid x$$

Lambda Calculus (Semantics)

$$\frac{e_1 \longrightarrow e_2}{e_1 e \longrightarrow e_2 e}\ \textsc{appLeft} \qquad \frac{e_1 \longrightarrow e_2}{e e_1 \longrightarrow e e_2}\ \textsc{appRight} \qquad \frac{e_1 \longrightarrow e_2}{\lambda x.e_1 \longrightarrow \lambda x.e_2}\ \textsc{fun}$$

$$\frac{}{(\lambda x.e_1)e_2 \longrightarrow [e_2/x]e_1}\ \textsc{beta}$$

$$v \ ::= \ \lambda x.v \mid x v \ldots v$$

Capture-Avoiding Substitution

$$[e/y]x = \begin{cases} e & x = y \\ x & \texttt{else} \end{cases}$$

$$[e/y](\lambda x\,.\,e') = \begin{cases} \lambda x\,.\,e' & x = y \\ \lambda z\,.\,[e/y][z/x]e' & x \in FV(e), z \texttt{ is fresh} \\ \lambda x\,.\,[e/y]e' & \texttt{else} \end{cases}$$

$$[e/y](e_1 e_2) = ([e/y]e_1)([e/y]e_2)$$

α-Equivalence

```
let x = 2 in x + 1
```

$=_\alpha$

```
let z = 2 in z + 1
```

OCaml

$$\lambda x\,.\,\lambda y\,.\,x =_\alpha \lambda v\,.\,\lambda w\,.\,v$$

λ-calculus

Evaluation Strategies

Non-Capture-Avoiding Substitution

$$[e/y]x = \begin{cases} e & x = y \\ x & \texttt{else} \end{cases}$$

$$[e/y](\lambda x\,.\,e') = \begin{cases} \lambda x\,.\,e' & x = y \\ \lambda x\,.\,[e/y]e' & \texttt{else} \end{cases}$$

$$[e/y](e_1 e_2) = ([e/y]e_1)([e/y]e_2)$$

Non-Capture-Avoiding Substitution

$$[e/y]x = \begin{cases} e & x = y \\ x & \texttt{else} \end{cases}$$

$$[e/y](\lambda x\,.\,e') = \begin{cases} \lambda x\,.\,e' & x = y \\ \lambda x\,.\,[e/y]e' & \texttt{else} \end{cases}$$

$$[e/y](e_1 e_2) = ([e/y]e_1)([e/y]e_2)$$

Thm. If e is closed then this definition is equivalent to capture avoiding substitution

Evaluation under binders

$$\frac{e_1 \longrightarrow e_2}{e_1 e \longrightarrow e_2 e}\ \textsc{appLeft} \qquad \frac{e_1 \longrightarrow e_2}{e e_1 \longrightarrow e e_2}\ \textsc{appRight}$$

$$\frac{}{(\lambda x.e_1)e_2 \longrightarrow [e_2/x]e_1}\ \textsc{beta}$$

$$v \ ::= \ \lambda x.e$$

If we remove the (fun) rule, then we can use non-capture avoiding substitution for closed expressions

A Small-Step Semantics

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2} \qquad (2)\ \frac{e_2 \longrightarrow e_2'}{(\lambda x\,.\,e_1)e_2 \longrightarrow (\lambda x\,.\,e_1)e_2'}$$

$$(3)\ \frac{}{(\lambda x\,.\,e)(\lambda y\,.\,e') \longrightarrow [(\lambda y\,.\,e')/x]e}$$

A Small-Step Semantics

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}$$

$$(2)\ \frac{e_2 \longrightarrow e_2'}{(\lambda x\,.\,e_1)e_2 \longrightarrow (\lambda x\,.\,e_1)e_2'}$$

$$(3)\ \frac{}{(\lambda x\,.\,e)(\lambda y\,.\,e') \longrightarrow [(\lambda y\,.\,e')/x]e}$$

1. We can reduce the LHS of an application

A Small-Step Semantics

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}$$

$$(2)\ \frac{e_2 \longrightarrow e_2'}{(\lambda x\,.\,e_1) e_2 \longrightarrow (\lambda x\,.\,e_1) e_2'}$$

$$(3)\ \frac{}{(\lambda x\,.\,e)(\lambda y\,.\,e') \longrightarrow [(\lambda y\,.\,e')/x]e}$$

1. We can reduce the LHS of an application
2. We can reduce the RHS of an application *if the LHS is already a function*

A Small-Step Semantics

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}$$

$$(2)\ \frac{e_2 \longrightarrow e_2'}{(\lambda x\,.\,e_1)e_2 \longrightarrow (\lambda x\,.\,e_1)e_2'}$$

$$(3)\ \frac{}{(\lambda x\,.\,e)(\lambda y\,.\,e') \longrightarrow [(\lambda y\,.\,e')/x]e}$$

1. We can reduce the LHS of an application
2. We can reduce the RHS of an application *if the LHS is already a function*
3. We can apply a function to value via β

Another Small-Step Semantics

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}$$

$$(2)\ \frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}$$

Another Small-Step Semantics

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}$$

$$(2)\ \frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}$$

1. We can reduce the LHS of an application

Another Small-Step Semantics

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}$$

$$(2)\ \frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}$$

1. We can reduce the LHS of an application
2. We can apply a function to *any* expression via substitution

A Big-Step Semantics

$$(1)\ \frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$(2)\ \frac{e_1 \Downarrow \lambda x\,.\,e \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

A Big-Step Semantics

$$(1)\ \frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$(2)\ \frac{e_1 \Downarrow \lambda x\,.\,e \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

1. A function evaluates to a function value (itself)

A Big-Step Semantics

$$(1)\ \frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$(2)\ \frac{e_1 \Downarrow \lambda x\,.\,e \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

1. A function evaluates to a function value (itself)

2. If e_1 evaluates to the function $\lambda x\,.\,e$ and e_2 evaluates to the value v_2 and e with v_2 substituted for x evaluates to v, then the application $e_1 e_2$ evaluates to v

Another Big-Step Semantics

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e \qquad [e_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

These are the same rules as before except we're not required to evaluate e_2 first

Call-by-Value vs. Call-by-Name

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\,\text{(CBV)}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\,\text{(CBN)}$$

Call-by-Value vs. Call-by-Name

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\,\text{(CBV)}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\,\text{(CBN)}$$

The two versions of semantics we've given correspond to **call-by-value** (CBV) and **call-by-name** (CBN)**.** These are **evaluation strategies** for functional PLs

Call-by-Value vs. Call-by-Name

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\,\text{(CBV)}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\,\text{(CBN)}$$

The two versions of semantics we've given correspond to **call-by-value** (CBV) and **call-by-name** (CBN)**.** These are **evaluation strategies** for functional PLs

CBV: evaluate the argument of a function *before* substituting it in the function

Call-by-Value vs. Call-by-Name

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\,\text{(CBV)}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\,\text{(CBN)}$$

The two versions of semantics we've given correspond to **call-by-value** (CBV) and **call-by-name** (CBN)**.** These are **evaluation strategies** for functional PLs

CBV: evaluate the argument of a function *before* substituting it in the function

CBN: substitute the expression *directly* into the function

Benefits of CBV

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,x + x + x + x)e$$

Benefits of CBV

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,x + x + x + x)e$$

If we compute the value of an argument before substituting it into the expression, we only have to compute the expression *once*

Benefits of CBV

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,x + x + x + x)e$$

If we compute the value of an argument before substituting it into the expression, we only have to compute the expression *once*

This is good if the variable appears several times in the body of our function

Benefits of CBV

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,x + x + x + x)e$$

If we compute the value of an argument before substituting it into the expression, we only have to compute the expression *once*

This is good if the variable appears several times in the body of our function

This is also called **eager**, or **applicative**, or **strict** evaluation (and is what OCaml does)

Benefits of CBN

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,\lambda y\,.\,x)e_1 e_2$$

Benefits of CBN

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,\lambda y\,.\,x)e_1e_2$$

If a variables doesn't appear in our function, then the argument is *not evaluated at all*

Benefits of CBN

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,\lambda y\,.\,x)e_1e_2$$

If a variables doesn't appear in our function, then the argument is *not evaluated at all*

If an argument is only seldomly used, it will only be computed when it is used (e.g, if its computed in a branch of an if-expression that is almost never reached)

Benefits of CBN

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,\lambda y\,.\,x)e_1e_2$$

If a variables doesn't appear in our function, then the argument is *not evaluated at all*

If an argument is only seldomly used, it will only be computed when it is used (e.g, if its computed in a branch of an if-expression that is almost never reached)

Aside. It's possible to simulate CBN in CBV. Think about it for a bit, ask me after if you're interested

Aside: Evaluation Strategies

Aside: Evaluation Strategies

There are *a lot* more evaluation strategies, all of which optimize *something*

Aside: Evaluation Strategies

There are *a lot* more evaluation strategies, all of which optimize *something*

Haskell uses **lazy** evaluation also called **call-by-need**

Aside: Evaluation Strategies

There are *a lot* more evaluation strategies, all of which optimize *something*

Haskell uses **lazy** evaluation also called **call-by-need**

In languages with pointers we also often have the option to use **call-by-reference** evaluation or **call-by-sharing**

workshop